

MODÈLES DE MARKOV CACHES ET INTÉGRATION IA EN FINANCE

Rapport de stage

Auteur : Paul de Pazanan

github.com/PdePazanan/Internship_SNU

Tuteur de stage : Pr. Tae-Wan Kim

Contents

1	Remerciement	3
2	Abstract	4
3	Introduction	5
3.1	L'équipe	7
3.2	Les outils utilisés	8
3.2.1	Linux	8
3.2.2	GitHub	8
3.2.3	Latex	9
3.2.4	Scikit-learn	9
3.2.5	Optuna	9
4	Le travail réalisé	10
4.1	Objectif	10
4.2	Modeles de Markov	11
5	Définitions	11
5.1	Chaine de Markov	11
5.2	Hidden Markov Model (HMM)	12
6	HMM classique	13
7	Observations du marché	14
8	Détermination des paramètres	16
9	Forward algorithm	16
10	HMM Gaussien	17
10.1	Expectation	17
10.2	Maximization	17
11	Variational Bayesian Inference (approximation champ moyen)	19
11.1	L'intégrale impossible	19
11.2	Étape V-E (Variational Expectation)	20
11.3	Étape V-M (Variational Maximization)	20
11.4	Priors	20
12	Détection des états	22
13	Implémentation avec des réseaux de neurones	24
14	Utilisation d'optuna pour optimiser l'entraînement	26
15	Entraînement	28
16	Resultats	29

17 Performances de la strategies en utilisant l'implementation aux reseaux de neurones	30
18 Performances de la stratégie utilisant seulement le modèle de Markov caché	31
19 Travail en temps reel	32
19.1 API TradeStation	32
19.2 Implémentation en temps réel	33
20 Travail Future	34
21 Conclusion	35
22 References	38

1 Remerciement

Je tiens à remercier, pour ce stage, l'INSA Lyon qui a publié l'offre partenaire et qui m'a permis de partir à l'étranger pour réaliser ce stage au laboratoire de Seoul National University.

Je remercie également le professeur Tae-Wan Kim, qui m'a accompagné en tant que tuteur de stage durant toute la durée de mon expérience. Je remercie également toute l'équipe du laboratoire, avec qui j'ai pu découvrir le monde du travail ainsi que la culture coréenne.

2 Abstract

Les modèles de Markov cachés constituent une classe polyvalente de modèles statistiques pour différentes données et particulièrement les séries temporelles. C'est une méthode de machine learning puissante pour la détection de régimes dans une classe de données. Combiner ces modèles de Markov à des réseaux de neurones permet également d'augmenter la précision des détections.

Dans les applications financières ces régimes peuvent souvent être liés à des marchés baissiers ou haussiers, provoquant donc des opportunités de trading.

Durant ce stage au laboratoire de Deep Learning de Seoul National University, nous avons eu la chance d'étudier ce sujet passionnant et de développer différentes méthodes liées aux modèles cachés de Markov afin de créer un agent de trading en temps réel capable de se positionner sur un marché en fonction du régime actuel.

3 Introduction

Le laboratoire est situé à l'Université nationale de Séoul (SNU), dans le quartier de Gwanak, au sud de Séoul, en Corée. L'université est organisée en différents départements de recherche et d'enseignement. Ces départements sont regroupés au sein de différentes facultés, telles que la Faculté des sciences naturelles, axée sur la physique, la chimie, la biologie et les mathématiques ; la Faculté des sciences sociales, dédiée à des domaines comme l'économie, la sociologie et les sciences politiques ; et la Faculté des lettres et sciences humaines, qui étudie les langues, la littérature et la philosophie.



Figure 1: Map de SNU

De plus, SNU possède des écoles spécialisées telles que la Faculté de médecine, la Faculté d'administration des affaires et la Faculté des sciences de l'éducation, qui proposent différentes formations à la recherche. La SNU a le privilège d'abriter l'une des plus grandes bibliothèques universitaires d'Asie, qui compte plus de 5,2 millions de livres, 260 000 revues académiques et 230 000 ressources numériques.

L'Institut d'études coréennes Kyujanggak, fondé à l'origine comme bibliothèque royale de la dynastie Joseon, est géré par l'Université nationale de Séoul (SNU) et constitue un centre majeur de recherche en histoire coréenne.

Depuis 2022, la SNU compte environ 28 264 étudiants, dont 15 870 en licence, 12 394 en master et 4 019 en doctorat. L'université emploie environ 2 278 enseignants-chercheurs,

sous la direction du président Ryu Hong-lim. La SNU est répartie sur quatre campus principaux :

- **Campus de Gwanak** (campus principal) - situé à Gwanak-gu, Séoul
- **Campus de Yeongeon** - Facultés de médecine, d'odontologie et de sciences infirmières
- **Campus de Pyeongchang** - spécialisé en agriculture et sciences de la vie
- **Campus de Siheung** - un pôle de recherche et d'innovation mondial en développement



Figure 2: Campus de Gwanak

Ensemble, ces campus couvrent près de 7 km², faisant de l'Université nationale de Séoul (SNU) l'une des plus grandes universités de Corée du Sud. L'université compte 16 facultés, une école doctorale générale proposant 99 programmes et 12 écoles doctorales professionnelles.

Les principales facultés comprennent la Faculté des lettres, des sciences sociales, des sciences naturelles, de l'administration des affaires, de l'éducation, de la médecine, de la pharmacie, de l'agriculture et des sciences de la vie, des beaux-arts, de la musique et la Faculté d'ingénierie.

Cette dernière joue un rôle central dans la promotion de l'innovation technologique et comprend des départements tels que le génie mécanique, le génie électrique et informatique, le génie civil et environnemental, le génie chimique et biologique et la science et le génie des matériaux.

La recherche menée au sein de la Faculté d'ingénierie intègre des principes scientifiques avancés à des applications pratiques visant à relever les défis du monde réel. Parmi tous ces départements, le stage s'est déroulé au sein de la Faculté d'Ingénierie,

spécialisée dans l'innovation scientifique et technologique. Cette faculté regroupe plusieurs départements, notamment :

Génie mécanique, Génie électrique et informatique, Génie civil et environnemental, Génie chimique et biologique, Science et génie des matériaux, et bien d'autres.

Ces départements sont fortement impliqués dans des projets d'ingénierie et de recherche, contribuant de manière significative au développement industriel et sociétal.



Figure 3: Batiment du laboratoire

Le laboratoire où j'ai effectué mon stage est le Deep Learning Lab (DLLab), situé dans les bâtiments d'ingénierie de la Faculté d'ingénierie de l'Université nationale de Séoul.

Le laboratoire est dirigé par le professeur Kim Tae-Wan, qui a été mon tuteur en entreprise.

Fondé en 2007 sous la direction du professeur Kim, le laboratoire a bâti sa réputation sur des recherches de haut niveau en apprentissage profond, notamment en vision par ordinateur, en planification de trajectoires pour les systèmes autonomes et en applications robotiques pour l'ingénierie maritime et océanique.

Le domaine du laboratoire s'étend au-delà des contextes navals et océaniques : les recherches actuelles portent sur les modèles de langage à grande échelle (LLM), l'apprentissage par renforcement, la localisation et la cartographie simultanées (SLAM), l'inférence en périphérie, le matériel et les initiatives de collaboration entre start-ups et entreprises.

3.1 L'équipe

L'équipe du laboratoire était principalement composée de doctorants, étudiants, stagiaires et ingénieur. Le directeur de ce laboratoire est le professeur Tae-Wan Kim, qui agissait en tant que notre superviseur. Il était chargé de fournir des conseils et des commentaires sur notre travail hebdomadaire et l'orientation de nos recherches. Dans l'espace de travail, Jae Hyun Kim, un Ph.D. étudiant, était le responsable du laboratoire. Il a supervisé l'entretien du laboratoire et la gestion quotidienne des tâches et besoins des travailleurs pour leurs projets. Les autres membres du laboratoire ont travaillé dans divers domaines de recherches spécialisés, contribuant à un environnement diversifié et collaboratif.

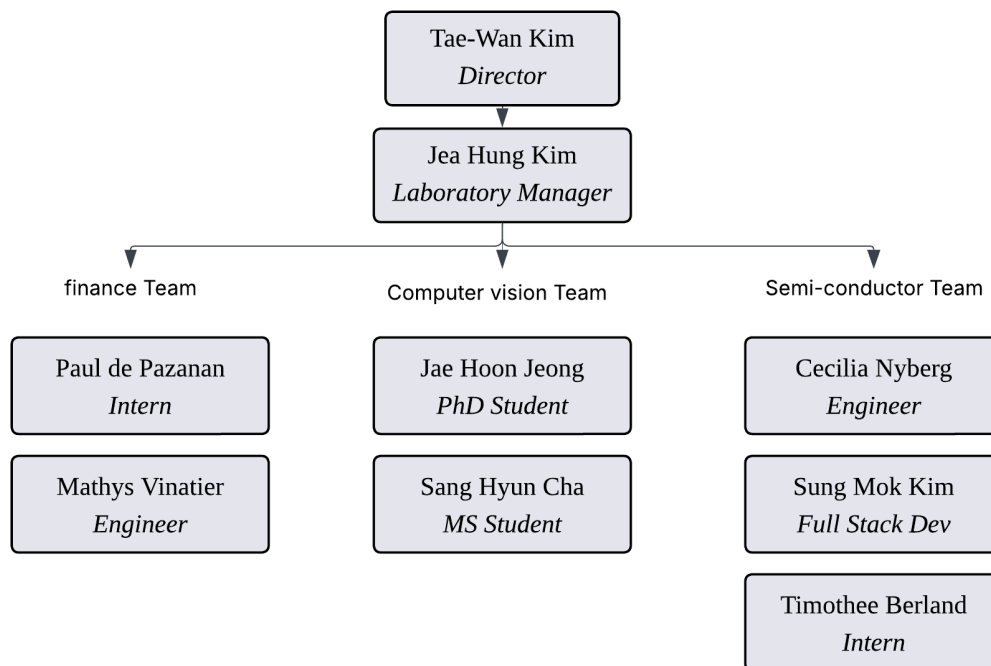


Figure 4: Organisation du laboratoire

L'équipe de vision par ordinateur était chargée de développer un modèle de langage étendu (LLM) multi-agents et un système de localisation et de cartographie simultanées (SLAM) pour la reconnaissance d'objets dans les environnements maritimes afin d'assurer une navigation sécuritaire. Cette équipe effectuait régulièrement des tests sur le terrain pour évaluer son matériel et ses applications. Ils possédaient une forte expertise en Deep Learning et en IA et échangeait souvent des idées avec l'équipe financière à travers des discussions sur la manière dont l'IA pourrait être introduite et ses limites.

L'équipe des semi-conducteurs s'est concentrée sur le développement de matériel pour l'inférence de l'IA, en particulier pour les agents basés sur LLM. Leur mission était de concevoir des systèmes embarqués basés sur FPGA capables d'exécuter des modèles informatiques lourds efficacement. Les ingénieurs de cette équipe ont travaillé sur la compilation et l'optimisation de modèles d'apprentissage profond comme des applications DeepSeek de chatbot afin que les cartes FPGA puissent les exécuter indépendamment.

avec un nombre limité de ressources informatiques.

Pour ma part, je faisais partie de l'équipe financière on mon principale rôle était de filtrer et analyser le marche pour permettre d'obtenir different régimes de marche et d'ensuite pouvoir implémenter un agent de prédiction en ajustant dynamiquement les filtres et les indicateurs pour automatiser les décisions de trading à l'aide de scripts. Je me suis également concentré sur les techniques de traitement du signal et de lissage afin d'identifier les solutions les plus adaptées. Un ingénieur de l'équipe s'est charge de développe un algorithme de Proximal Policy Optimization (PPO) Pour que je puisse implémenter mon HMM en tant qu'acteur

3.2 Les outils utilisés

Durant ce stage, nous avons utilise différents outils pour développer cet algorithme et pour le simuler. Nous avons utilise le langage de programmation Python pour écrire tout nos codes, ainsi que d'autres outils.

3.2.1 Linux

Durant le stage, nous avons utilisé le système d'exploitation Linux pour travailler aussi bien sur des postes de travail que sur des serveurs distants. Tout au long du projet, nous avons utilisé Linux Ubuntu comme environnement de bureau principal et des serveurs Ubuntu pour gérer les différents services et sessions de formation nécessitant l'exécution sur l'infrastructure du laboratoire. Je connaissait Linux très rapidement mais je ne l'avais jamais utilisé de manière aussi intensive sur un serveur auparavant.

Lors du développement du projet de stage, Linux s'est avéré extrêmement adapté, car il nous a permis de gérer très efficacement les ressources de l'ordinateur, notamment en ce qui concerne l'utilisation du GPU, l'allocation des processus et la surveillance du système. De plus, la plupart des outils de développement d'intelligence artificielle sont optimisés pour les systèmes Linux, ce qui a grandement facilité le débogage des problèmes de bas niveau et la résolution des problèmes d'environnement.

3.2.2 GitHub

Nous avons utilisé Git tout au long de ce stage, notamment pour conserver une copie fiable de notre travail et pour faciliter l'accès à nos scripts à des fins collaboratives. Nous avons également utilisé Git pour publier automatiquement nos modèles d'apprentissage profond afin d'assurer leur sauvegarde continue, car leur entraînement était extrêmement long et nous ne pouvions pas nous permettre de perdre la moindre progression.

3.2.3 Latex

Nous avons utilise Latex a des fins de rédactions pour nos rapports, il fallait en continue écrire et résume le travail que nous réalisons pour que le manager puisse nous guider adéquatement sur le travail a effectue ensuite.

3.2.4 Scikit-learn

Scikit-learn est la bibliothèque standard de l'industrie pour le Machine Learning (apprentissage automatique) en Python. Elle est construite sur d'autres bibliothèques scientifiques performantes comme NumPy, SciPy et Matplotlib. Elle permet de fournir des outils simples et efficaces pour l'analyse de données et la modélisation.



Figure 5: Logo sklearn

Les fonctionnalites cles de cette bibliotheque sont : la classification, régression, clustering (regroupement), réduction de dimensionnalité et prétraitement des données. Cette bibliothèque est populaire car qu'on utilise une régression linéaire ou un réseau de neurones complexe, les commandes sont les mêmes : `.fit()` et `.predict()`.

3.2.5 Optuna



Figure 6: Logo optuna

Optuna est une bibliothèque Python utilisée pour l'optimisation des tâches. Nous l'avons utilisé pour affiner les hyperparamètres de notre modèle. Avec Optuna, nous pouvons créer une boucle automatisée qui exécute notre modèle formation tout en sauvegardant les différents résultats et métriques d'évaluation de chaque modèle dans une base de données. Optuna dispose également de sa propre base de données interne, où il utilise des algorithmes de recherche avancés, tels que la recherche par grille et méthodes d'échantillonnage plus sophistiquées, pour explorer efficacement l'espace et converger vers l'optimisation définie

4 Le travail réalisé

4.1 Objectif

Nous avons vu les outils utilisés et une introduction à l'entreprise. Concentrons-nous maintenant sur le travail concrètement réalisé.

Le but du stage était de pouvoir fournir des signaux fiables pour du trading algorithmique sur un marché très précis du VIX Index, et de pouvoir trader cet indice à l'aide de [Futures](#). La figure ci-dessous nous montre le prix du VIX Index depuis 2000. Nous pouvons voir que son prix oscille entre 0 et 100, car cet indice mesure la volatilité attendue sur 30 jours du S&P 500, ou autrement dit la peur des investisseurs. Plus le prix est élevé, plus la volatilité est haute, ce qui représente une peur élevée chez les investisseurs.

Nous allons donc développer un outil en deux parties permettant de se positionner sur ce marché. Premièrement, développer un algorithme permettant de filtrer le marché et d'analyser dans quel régime nous sommes présents (régime haussier, baissier ou neutre). Puis, il faudra intégrer le modèle créé à des réseaux de neurones pour améliorer la précision des détections et produire des signaux d'achat ou de vente fiables.

De plus, il nous faudra également développer différents outils de simulation permettant de tester nos méthodes en tant réel, car le but final est de pouvoir déployer notre algorithme sur un marché réel.

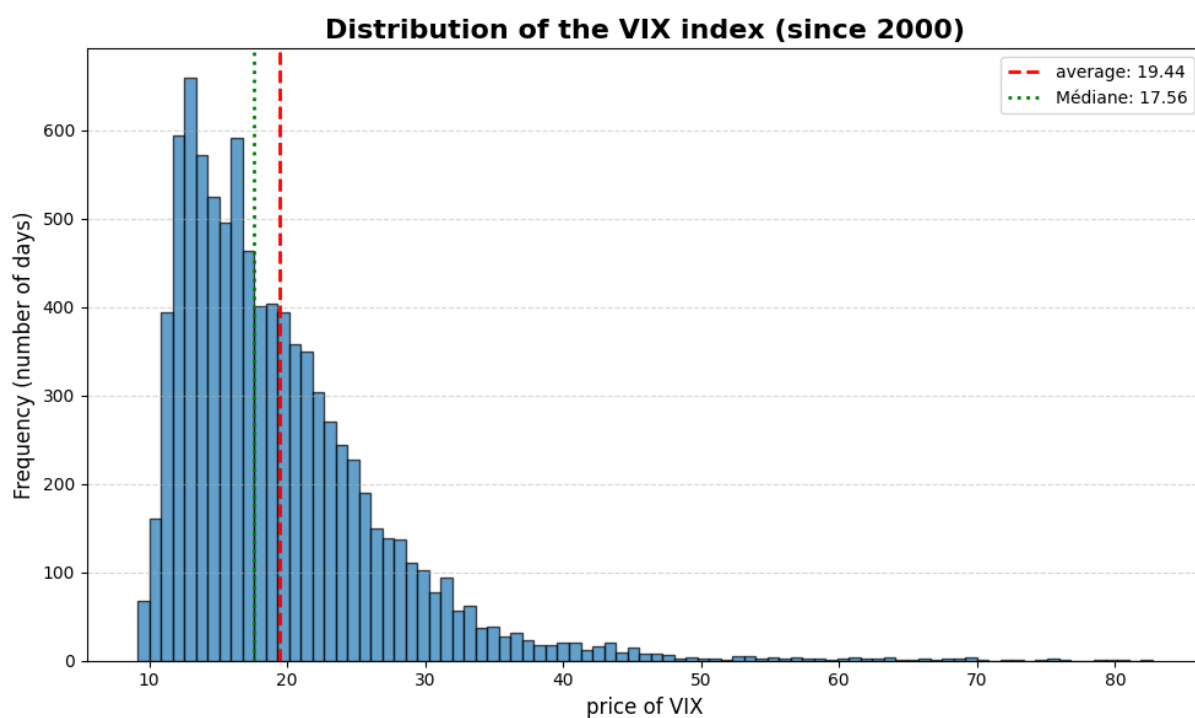


Figure 7: Distribution des prix du VIX depuis 2000

4.2 Modeles de Markov

Pour réaliser cela, nous avons utilisé des modèles de Markov cachés ou Hidden Markov Model (HMM).

En effet, ce choix apparaissait comme pertinent, car ces modèles constituent une classe polyvalente de modèles statistiques pour différentes données, et particulièrement les séries temporelles. Les données de marché sont des séries temporelles, et parfois très bruitées, ce qui rend, à l'œil nu, difficile la détection de régimes apparents. C'est pour cela que Les HMM sont apparu comme un choix naturel permettant de déterminer différents états dans un marché financier stationnaire (Le VIX index) ou les états peuvent être compliqués à repérer manuellement.

5 Définitions

5.1 Chaîne de Markov

Afin de comprendre les modèles de Markov cachés (HMM), il est d'abord nécessaire d'étudier les chaînes de Markov. Une chaîne de Markov est un modèle stochastique qui représente une suite d'événements ou d'états. Soit S un ensemble fini ou dénombrable et (X_n) un processus stochastique à valeurs dans S . Alors X est une chaîne de Markov si, pour tout $n \in \mathbb{N}$ et pour tous $x_0, x_1, \dots, x_{n+1} \in S$ tels que $P(X_0 = i_0, \dots, X_n = i_n) > 0$, on a

$$P(X_{n+1} = i_{n+1} \mid X_0 = i_0, \dots, X_n = i_n) = P(X_{n+1} = i_{n+1} \mid X_n = i_n) \quad (1)$$

Cela signifie que la probabilité de transition vers un certain état à l'instant $n + 1$ ne dépend que de l'état courant à l'instant n et non des états précédents.

Une chaîne de Markov est définie par l'ensemble des états S et par une matrice de probabilités de transition $A = [a_{ij}]$, où a_{ij} représente la probabilité de passer de l'état S_i à l'état S_j en une seule étape.

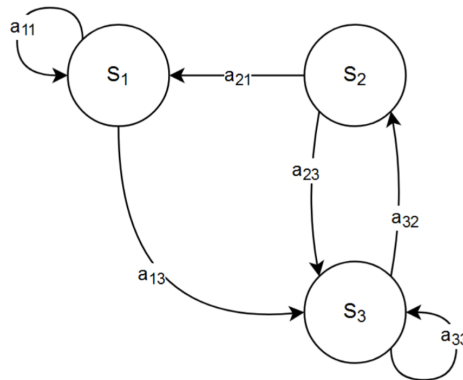


Figure 8: Chaîne de Markov à 3 états

Soit $x_n \in \mathbb{R}^N$ le vecteur contenant les probabilités d'être dans chacun des états à l'instant n . L'évolution du système est alors donnée par

$$x_{n+1}^T = x_n^T A \quad (2)$$

5.2 Hidden Markov Model (HMM)

Contrairement à une chaîne de Markov classique où les états sont visibles et observables, dans un modèle de Markov caché (HMM), les états sont cachés et sont déduits à partir des observations. Le processus des états cachés d'un HMM est une chaîne de Markov.

Soit

$$O = \{O_k \mid O_k \in \mathcal{O}, k = 1, \dots, T\}$$

une séquence observée, où $\mathcal{O}$ désigne l'ensemble des observations possibles. Les états cachés évoluent selon l'équation (2).

La structure d'un HMM est définie par ses différentes caractéristiques :

- λ est l'ensemble des paramètres du modèle ;
- N est le nombre d'états cachés du modèle. L'ensemble des états cachés est $S = (S_1, \dots, S_N)$ et l'état caché à l'instant t est noté Y_t ;
- M est le nombre de symboles distincts observables par état. Ces symboles sont notés o_k avec $k = 1, 2, \dots, M$ et l'observation à l'instant t est notée O_t ;
- $A = \{a_{ij}\}$ est la matrice de transition des états cachés, définie par :

$$a_{ij} = P(Y_{t+1} = S_j \mid Y_t = S_i), \quad i = 1, 2, \dots, N, j = 1, 2, \dots, N;$$

- $B = \{b_i(o_k)\}$ est la matrice des probabilités d'observation du symbole o_k lorsque le système est dans l'état S_i , définie par :

$$b_i(o_k) = P(O_t = o_k \mid Y_t = S_i), \quad i = 1, 2, \dots, N, k = 1, 2, \dots, M;$$

- $\mu = \{\mu_i\}$ est la distribution de probabilité de l'état initial du modèle, définie par :

$$\mu_i = P(Y_1 = S_i), \quad i = 1, 2, \dots, N.$$

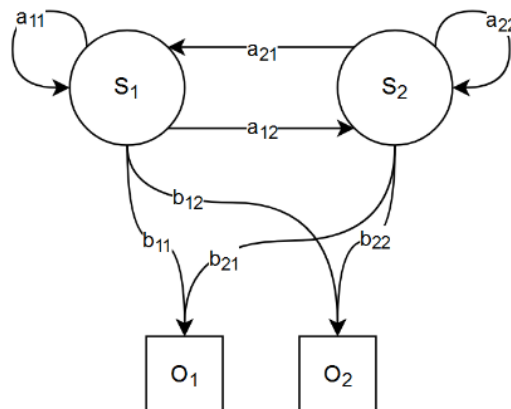


Figure 9: Hidden Markov model chain with 2 states and 2 possible outputs

6 HMM classique

Le HMM est donc une méthode pertinente pour détecter différents régimes dans un marché financier où les régimes ne sont pas toujours observables.

Nous avons utilisé l'indice VIX et défini un HMM à trois états, en émettant l'hypothèse que le VIX Index est stationnaire et qu'il oscillera toujours entre 0 et 100. Nous définissons ainsi un état normal, où l'indice se situe autour de son prix moyen sans opportunité d'achat ou de vente, un état bottom, où l'indice est survenu et provoque donc une opportunité d'achat, et enfin un état top, où l'indice est surévalué et provoque donc une opportunité de vente.

Comme nous l'avons vu précédemment, ces trois états sont cachés, car nous ne pouvons pas les déterminer directement : il nous faut des observations du marché. Ce processus d'états cachés est défini par une chaîne de Markov simple, comme le montre la figure 10 avec sa partie en rouge.

Le modèle prendra donc des observations du marché, que nous définirons plus tard, et qui permettront de déterminer nos états cachés, comme représenté sur la figure 10 avec la partie verte. Le modèle est composé de trois paramètres importants dont nous étudierons plus tard comment les trouver : une matrice de transition, qui représente la probabilité de passer d'un état à un autre au sein de la chaîne de Markov, ainsi que d'une matrice d'émission, qui représente les probabilités d'être dans un certain état étant donnée l'observation actuelle. Enfin, il y a les probabilités initiales, qui sont à déterminer.

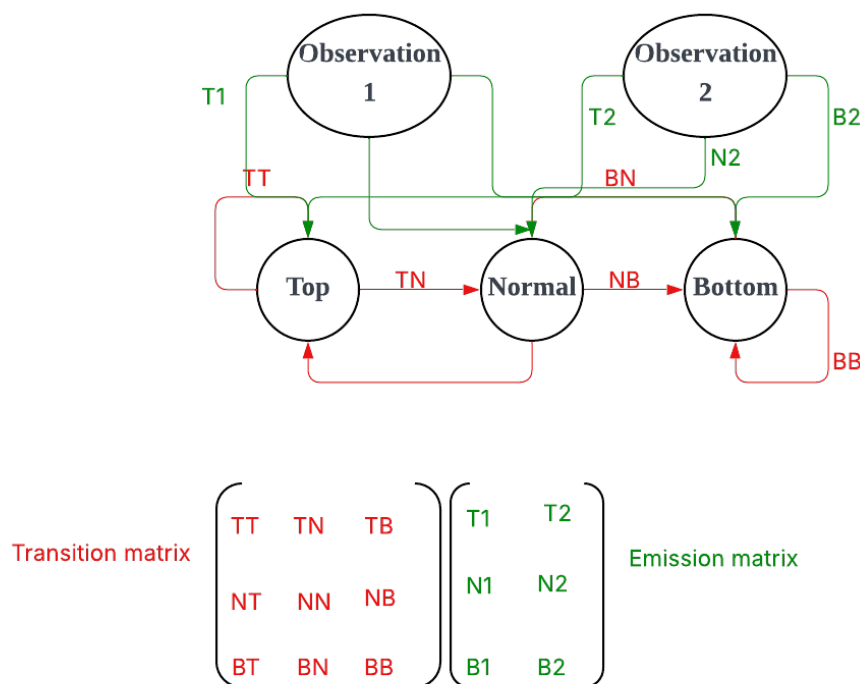


Figure 10: HMM a trois etats utilisant 2 observations

Le but du HMM est donc avec une suite d'observation de trouver la séquence d'état la plus probable. Comme représenté sur la figure 11, si nous avons la suite d'observations 1 puis 1 puis 2, il faut trouver la suite d'état la plus probable en utilisant la matrice d'émission et de transition. Le processus sera expliqué ultérieurement.

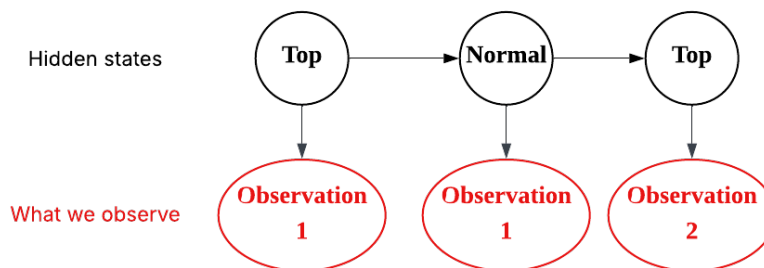


Figure 11: Hidden Markov model chain with 2 states and 2 possible outputs

7 Observations du marché

Pour pouvoir repérer nos différents régimes, nous avons donc besoin d'observations du marché. Sur le graphique précédent, deux observations étaient représentées ; il nous en faudra en réalité beaucoup plus pour gagner en précision. Cependant, il faudra également éviter d'en utiliser trop, car notre modèle pourrait ne pas repérer de patterns si trop de données lui sont fournies, et ainsi réaliser de l'overfitting.

Nous obtenons, à chaque période de temps t , la matrice suivante contenant les informations sur les prix du marche.

$$state = \begin{pmatrix} open_1 & close_1 & low_1 & high_1 & volume_1 \\ open_2 & close_2 & low_2 & high_2 & volume_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ open_t & close_t & low_t & high_t & volume_t \end{pmatrix}$$

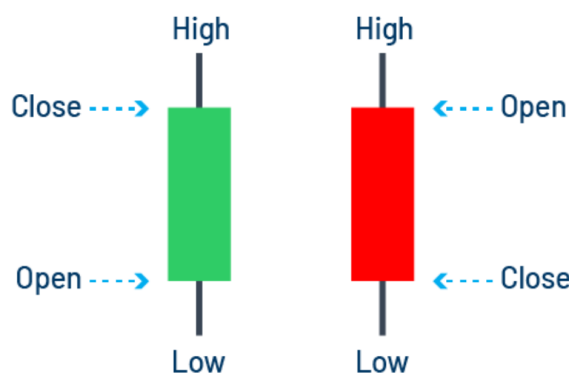


Figure 12: valeurs définissant une bougie

À partir de cette matrice, nous pouvons calculer des données utiles représentant l'état du marché ; nous appellerons ces données les observations. Ces observations peuvent être plus ou moins corrélées entre elles (figure 12) et sont déjà à l'origine de différentes stratégies de trading. Tout au long de notre étude, nous piocherons dans ces données et n'utiliserons pas toujours la totalité des observations. Les features les plus importantes sont les quatre suivantes :

1. **Log Returns** (Rendements logarithmiques) :

$$\text{logret}_t = \ln \left(\frac{P_t}{P_{t-1}} \right) \quad (1)$$

2. **Z-Score** (Normalisation sur 20 périodes) :

$$\text{zscore}_t = \frac{P_t - \mu_{20}}{\sigma_{20}} \quad (2)$$

Où μ_{20} est la moyenne mobile et σ_{20} l'écart-type sur 20 jours.

3. **Relative Range** (Volatilité relative) :

$$\text{range}_t = \frac{\text{High}_t - \text{Low}_t}{P_t} \quad (3)$$

4. **Slope10** (Pente du momentum) :

$$\text{slope10}_t = \frac{P_t - P_{t-10}}{10} \quad (4)$$

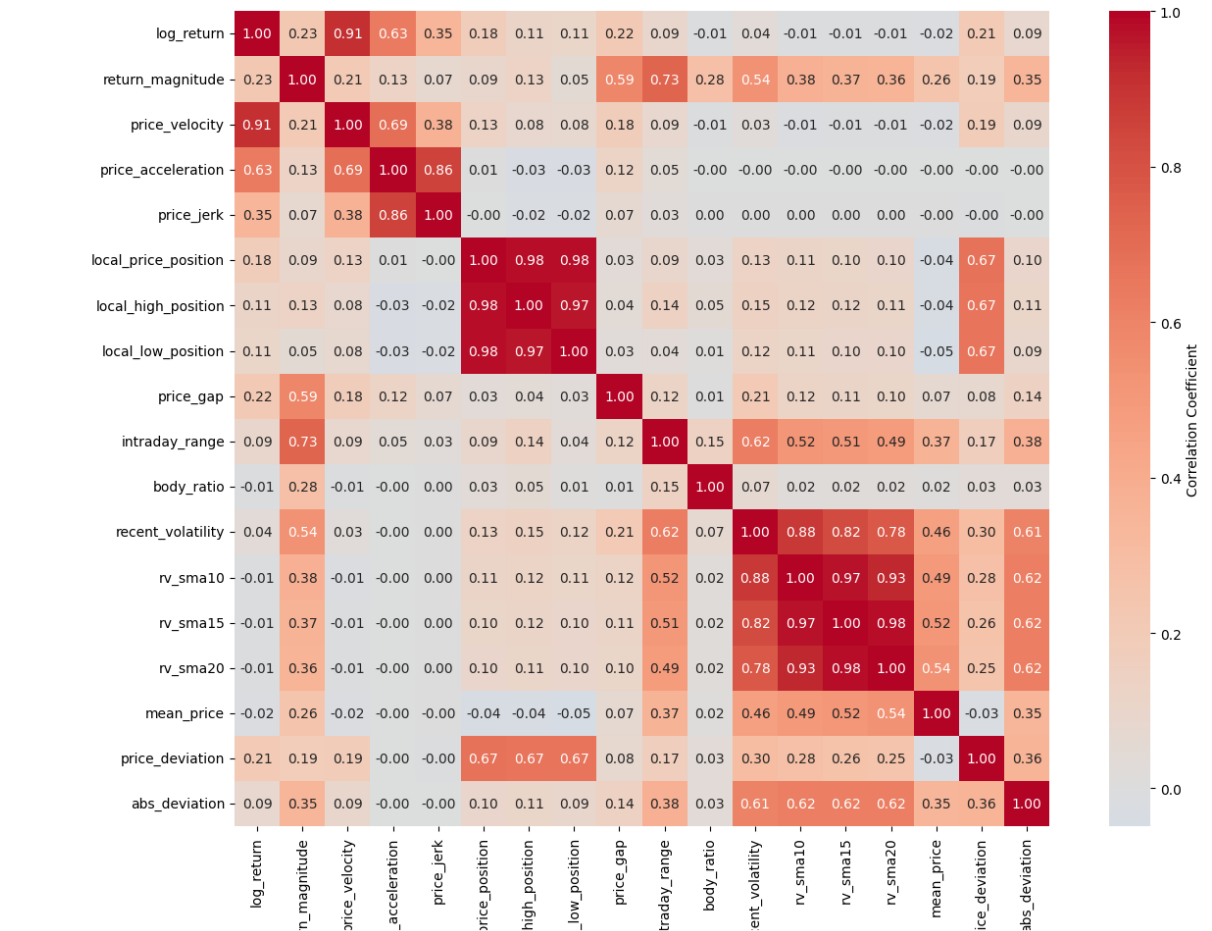


Figure 13: Matrice de corrélation des différentes observations

8 Détermination des paramètres

Nous avons vu la structure d'un modèle hmm et défini les observations qui nous permettent d'identifier les différents états. Il faut maintenant déterminer une méthode pour trouver ces variables. Il existe plusieurs chemins pour y parvenir :

- Utiliser des émissions heuristiques et définir nous-mêmes le HMM manuellement et les variables en utilisant les observations et en mettant à jour les paramètres à l'aide de Bayes update
- Utiliser la méthode classique avec l'algorithme de Baum-Welch et un modèle gaussien pour entraîner le modèle sur un dataset afin d'obtenir directement les paramètres
- Utiliser une méthode plus avancée avec le Variational Bayesian Inference

Nous cherchons à déterminer donc les 3 paramètres régissant le HMM:

- Les probabilités initiales
- la matrice de transition
- la matrice d'émission

9 Forward algorithm

Une fois que nos paramètres sont déterminés, il nous suffit d'appliquer un forward algorithm afin de pouvoir déterminer en temps réel l'état dans lequel nous sommes actuellement. La formule ci-dessous permet de résumer cela : à chaque instant de temps t , nous utilisons la matrice d'émission, la matrice de transition ainsi que les probabilités précédentes pour déterminer l'état actuel.

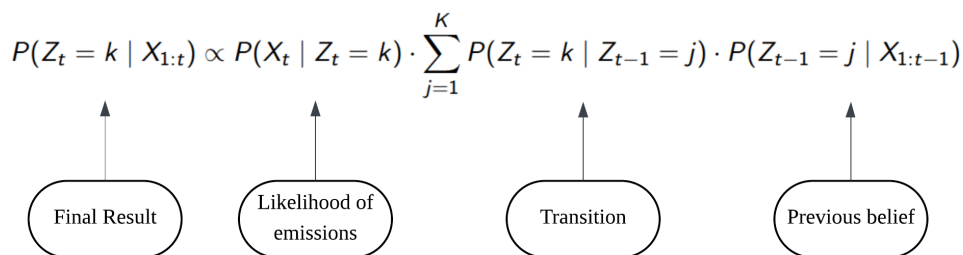


Figure 14: algorithme forward pour la période de test

10 HMM Gaussien

Une première méthode pour obtenir ces paramètres est d'appliquer le modèle simple d'un HMM gaussien, le plus répandu. Celui-ci nous retournera une valeur unique pour chaque probabilité de chaque paramètre. Il faut donc trouver les paramètres qui maximisent la probabilité d'observer notre séquence de données O . Pour cela, nous allons utiliser l'algorithme de Baum-Welch (ou Expectation Maximization Algorithm), qui fonctionne en deux étapes.

10.1 Expectation

Dans cette étape nous initialisons aléatoirement les paramètres que nous cherchons afin d'estimer la probabilité d'être dans un certain état à un instant donné.

Nous allons réaliser sur les données d'entraînement un algorithme forward-backward pour calculer 2 variables intermédiaires indispensables pour chaque instant t :

- $\gamma_t(i)$: La probabilité d'être dans l'état i à l'instant t , sachant toute la séquence d'observation.
- $\xi_t(i, j)$: La probabilité de passer de l'état i à l'instant t vers l'état j à l'instant $t + 1$.

10.2 Maximization

Maintenant que nous avons déterminé ces estimations, nous mettons à jour les paramètres du modèle pour qu'ils correspondent au mieux à ces estimations.

Les formules de mise à jour sont assez intuitives, puisque ce sont des moyennes pondérées :

Mise à jour de la transition(a_{ij}) :

$$\hat{a}_{ij} = \frac{\text{Espérance du nombre de transitions de } i \text{ vers } j}{\text{Espérance du nombre de transitions sortant de } i} = \frac{\sum_t \xi_t(i, j)}{\sum_t \gamma_t(i)}$$

Mise à jour de l'émission (cas discret)($b_j(k)$) :

$$\hat{b}_j(k) = \frac{\text{Espérance du nombre de fois où l'on est en } j \text{ et on observe } k}{\text{Espérance du nombre de fois où l'on est en } j}$$

L'explication ci-dessus suppose que les valeurs sont discrètes. Mais en réalité, dans la finance, les valeurs sont continues. Dans ce cas-là, la matrice d'émission B ne peut pas être seulement une table de probabilités. Nous remplaçons cette table par une fonction de densité de probabilité. La plus populaire est une gaussienne (loi normale).

Ainsi, pour l'étape de maximisation, nous devons apprendre les paramètres de la fonction gaussienne au lieu d'apprendre des probabilités discrètes :

- La Moyenne (μ_j) : Le "centre" des observations pour cet état.
- La Covariance (Σ_j) : La "dispersion" ou la forme du nuage de points pour cet état.

L'algorithme de Baum-Welch s'adapte très bien à cela et nous apprenons donc les nouveaux paramètres suivants :

Nouvelle Moyenne ($\hat{\mu}_j$), moyenne pondérée de toutes les observations o_t , où le poids est la probabilité que cette observation vienne de l'état j .

$$\hat{\mu}_j = \frac{\sum_t \gamma_t(j) \cdot o_t}{\sum_t \gamma_t(j)}$$

Nouvelle Covariance ($\hat{\Sigma}_j$), variance pondérée des observations par rapport à la nouvelle moyenne.

$$\hat{\Sigma}_j = \frac{\sum_t \gamma_t(j) \cdot (o_t - \hat{\mu}_j)(o_t - \hat{\mu}_j)^T}{\sum_t \gamma_t(j)}$$

Cet entraînement nous permettra de trouver des valeurs exactes pour nos paramètres. Le processus de l'algorithme est represente ci-dessous, figure 15 :

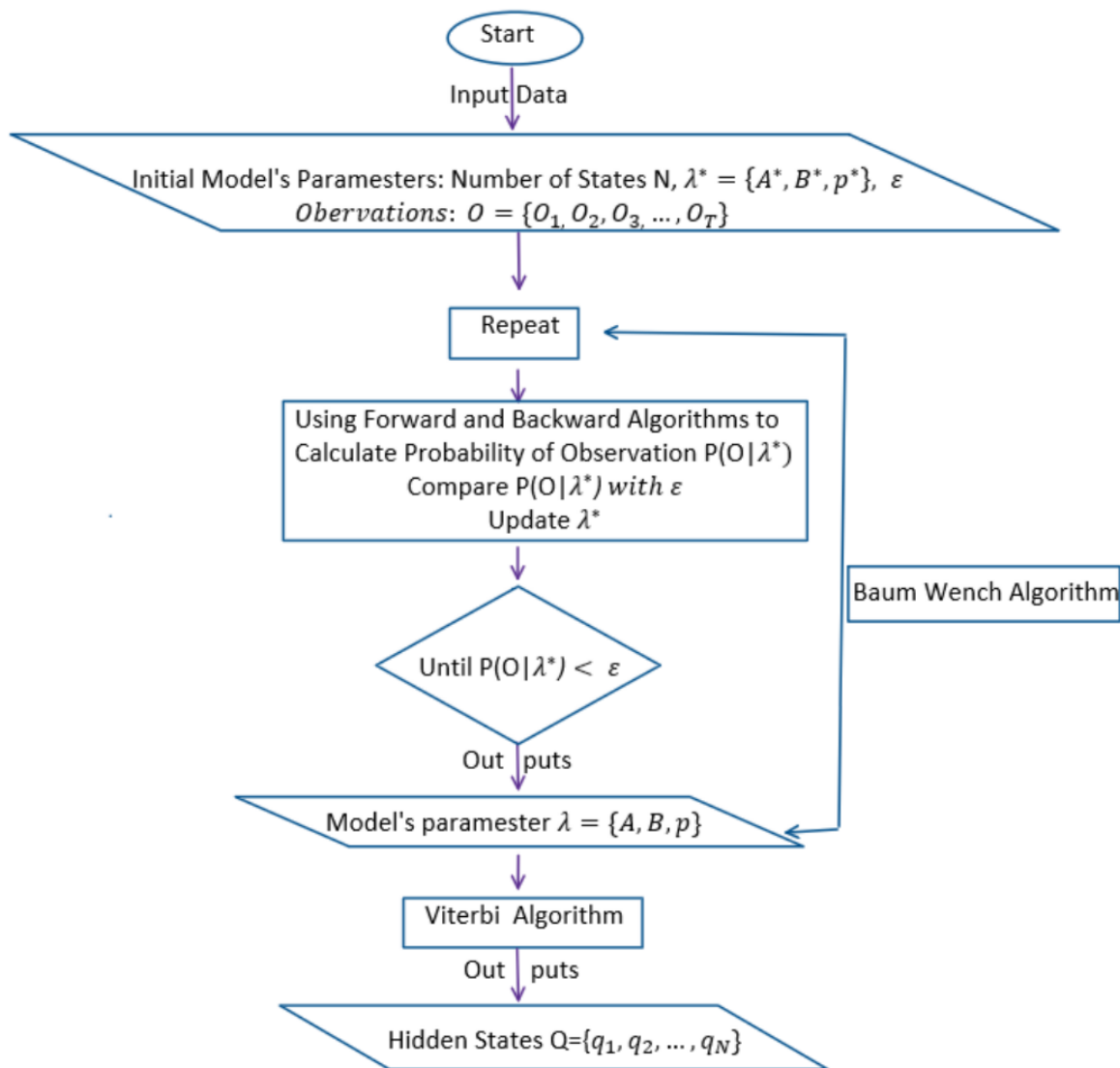


Figure 15: Processus d'entraînement et de détection du HMM

Cette méthode est simple à mettre en place, convergera toujours vers un optimum local, et peut s'adapter facilement aux émissions discrètes ou continues. Mais elle présente également des inconvénients que nous allons analyser dans la prochaine partie.

11 Variational Bayesian Inference (approximation champ moyen)

L'inconvénient le plus grand de la méthode classique de Baum-Welch est que les estimations données sont ponctuelles : l'algorithme nous donne une seule valeur précise pour chaque paramètre (ex : $\mu = 0.5$). Il ne nous donne aucune idée de l'incertitude sur cette valeur. En finance, ceci est un problème majeur, car si on a peu de données d'entraînement pour un certain état, notre modèle nous renverra une seule valeur et nous n'aurons pas d'informations sur le fait que cette valeur soit solide ou non.

De plus, si nous avons peu de données, le modèle va tout de même apprendre des paramètres, mais qui ne seront pas du tout précis, et cela va provoquer de l'overfitting, et les résultats en période de test seront bien moins bons.

Pour résoudre ce problème, il nous faut améliorer la précision des détectations. En effet, avec le HMM classique, pour passer de l'état 1 à 2, il y a une unique probabilité, par exemple 0,3. Mais si le modèle n'a pas eu le temps de bien apprendre, cette valeur ne sera pas réellement représentative. Ainsi, pour résoudre cela, nous allons utiliser un HMM bayésien, plus précisément un Variational Bayesian Inference. Cette deuxième méthode consiste, au lieu d'estimer une valeur ponctuelle unique, $\theta^* = \arg \max_{\theta} p(X | \theta)$, de chercher à approximer la distribution a posteriori complète des paramètres :

$$p(\theta | X) \approx q(\theta),$$

11.1 L'intégrale impossible

Pour trouver notre distribution, il nous faut rigoureusement appliquer la règle de Bayes, qui implique de calculer un dénominateur nécessitant une intégrale sur toutes les configurations possibles de paramètres. Le problème est que, dans un HMM complexe, cette intégrale est impossible à calculer. Nous allons donc utiliser une approche variationnelle.

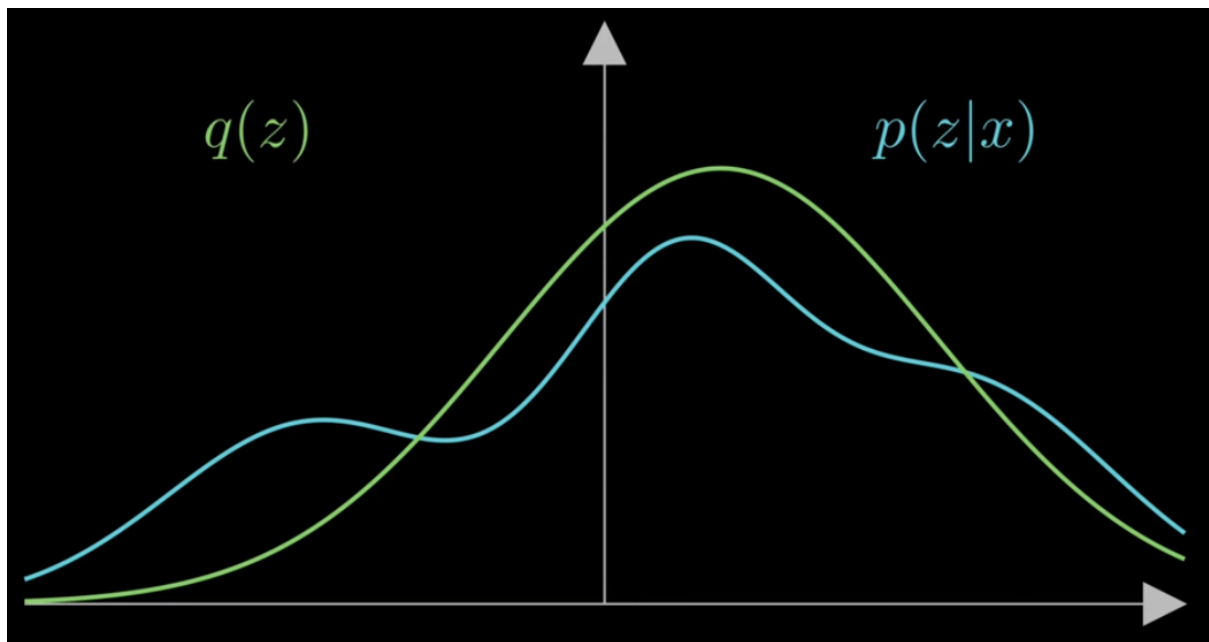


Figure 16: représentation de l'approche variationnelle

Comme nous le voyons sur la figure 16 au lieu de calculer la distribution précise $p(z|x)$ nous allons approximer cette distribution et trouvant $q(z)$.

Pour réaliser cela, nous allons utiliser un algorithme semblable a celui de Baum-Welch : le Variational Bayesian EM

11.2 Étape V-E (Variational Expectation)

Comme dans l'approche classique, on utilise l'algorithme Forward-Backward, mais au lieu d'utiliser les valeurs brutes des paramètres A et B , nous allons prendre leur espérance (la moyenne de leurs distributions actuelles).

On obtient donc les statistiques suffisantes (γ et ξ) qui nous disent quels états sont probables.

11.3 Étape V-M (Variational Maximization)

On met à jour les "hyper-paramètres" des distributions q .

Pour la matrice de transition, il est très populaire d'utiliser une loi de Dirichlet. On met à jour ses compteurs en ajoutant les transitions observées à l'étape E.

Pour les émissions Gaussiennes, on utilise une loi Normale-Wishart. On met à jour la moyenne et la matrice de précision en fonction des données pondérées.

11.4 Priors

Contrairement au HMM classique, nous plaçons donc des lois a priori conjuguées sur tous les paramètres du HMM ce qui n'était pas réalisé avant:

- Distribution initiale :

$$\pi \sim \text{Dir}(\alpha_0).$$

- Lignes de la matrice de transition :

$$A_i \sim \text{Dir}(\alpha_{A0}) \quad \text{pour tout } i.$$

- Lois d'émission gaussiennes (pour chaque état k et chaque dimension d) :

$$\tau_{kd} \sim \text{Gamma}(a_0, b_0) \quad (\text{précision}),$$

$$\mu_{kd} \mid \tau_{kd} \sim \mathcal{N}(\mu_0, (\lambda_0 \tau_{kd})^{-1}).$$

Après avoir utilisé l'algorithme en prenant les observations du marché définies précédemment, nous obtenons finalement les paramètres suivants pour notre modèle caché de Markov :

$$\text{Probabilités initiales } \boldsymbol{\pi} = (0.49990132 \quad 0.25006805 \quad 0.25003063)$$

$$\text{Matrice de transition } \mathbf{A} = \begin{pmatrix} 0.91315907 & 0.02261674 & 0.06422420 \\ 0.07877199 & 0.83853480 & 0.08269320 \\ 0.05266225 & 0.01839238 & 0.92894537 \end{pmatrix}$$

$$\text{Matrice d'émission } \boldsymbol{\mu}_{\text{mean}} (K \times D) = \begin{pmatrix} 8.660 \times 10^{-4} & 1.046 & 4.723 \times 10^{-3} & 1.629 \times 10^{-2} \\ 6.023 \times 10^{-4} & 0.1622 & 1.492 \times 10^{-2} & 1.158 \times 10^{-2} \\ -8.411 \times 10^{-4} & -1.037 & 4.210 \times 10^{-3} & -1.587 \times 10^{-2} \end{pmatrix}$$

$$\sqrt{\boldsymbol{\Sigma}_{\text{mean}}} (K \times D) = \begin{pmatrix} 0.00623040 & 0.73597400 & 0.00578912 & 0.01795274 \\ 0.02114252 & 1.77368075 & 0.01631164 & 0.15907400 \\ 0.00552426 & 0.70326405 & 0.00515582 & 0.01756078 \end{pmatrix}$$

En résumé, nous avons donc de meilleurs chances de performance en utilisant le Variational Bayesian Inference. Nous verrons plus tard que nos résultats étaient meilleurs avec le VC EM. Le tableau ci-dessous nous permet de résumer la différence entre ces deux méthodes :

Caractéristique	Baum–Welch (EM classique)	Variational Bayes (VB)
Objet recherché	Valeurs fixes (estimations ponctuelles)	Distributions de probabilité
Objectif mathématique	Maximisation de la vraisemblance	Maximisation de l'ELBO
Gestion du sur-apprentissage	Faible (risque élevé)	Excellente (régularisation intrinsèque)
Complexité	Simple, rapide	Plus complexe, mathématiquement lourde

Table 1: Comparaison entre Baum–Welch et Variational Bayes

12 Détection des états

Nous ajoutons maintenant à la détection de régimes une méthode pour pouvoir produire des signaux d'achat et de vente afin de regarder la robustesse de nos modèles. Nous introduisons un autre paramètre, qui est l'outlier line. En effet, à chaque instant t , notre modèle nous retourne une certaine probabilité d'être dans chaque état, mais pour avoir un signal d'achat, il ne faut pas forcément que ce soit celle qui a la plus grande probabilité : parfois, il ne faut juste rien faire. Ainsi, pour cela, nous utilisons cette outlier line qui fonctionne comme un threshold et qui représente un prix anormalement haut ou bas. Ainsi, si nos probabilités croisent cette ligne, cela provoquera un signal d'achat ou de vente, comme représenté sur la figure 17.

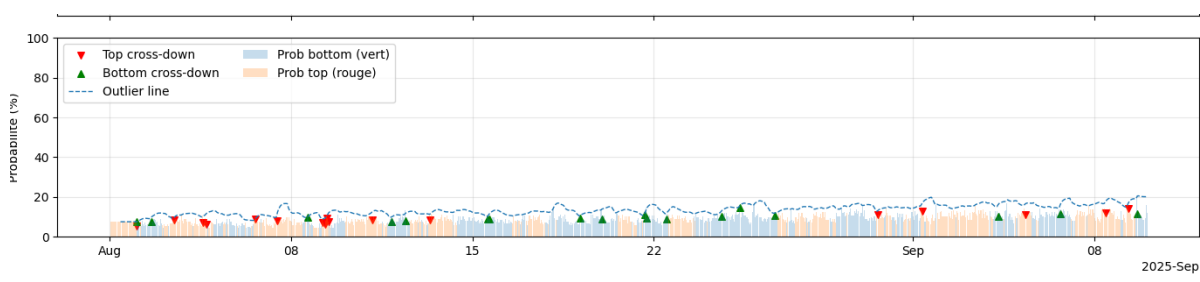


Figure 17: processus de déclenchement d'un signal

Notre modèle HMM classique permettait de bien repérer les états dans un grand intervalle de temps (journalier) : cela signifie que chaque bougie représente une journée. Mais lorsque nous descendions dans un intervalle de temps plus petit de 5 min (chaque bougie représente 5 minutes), alors le modèle n'arrivait plus à apprendre correctement et la détection devenait difficile. C'est pour cela que nous avons eu les résultats décevants de la figure 18 ci-dessous.

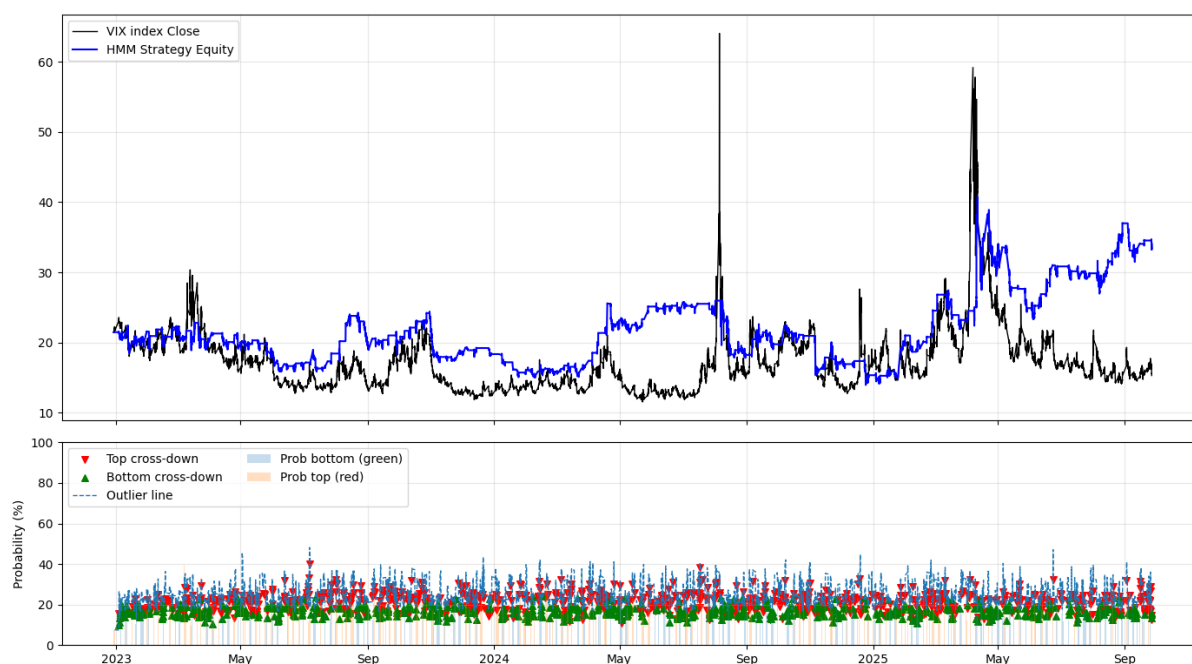


Figure 18: Resultats HMM Classique

Or, nous désirons travailler sur un petit intervalle de temps, car de gros frais s'appliquent en réalité si nous voulons tenir une position sur un [Futures](#) longtemps. En effet, chaque mois, le contrat expire et un nouveau est émis. Si nous voulons conserver la position, nous devons automatiquement vendre puis racheter, mais à un prix plus élevé, ce qui engendre une perte.

Ainsi, nous voyons que les résultats obtenus avec le HMM bayésien sur la figure ci-dessous sont bien meilleurs, même si d'importants [drawdowns](#) sont parfois présents ; ceci sera à résoudre ultérieurement.

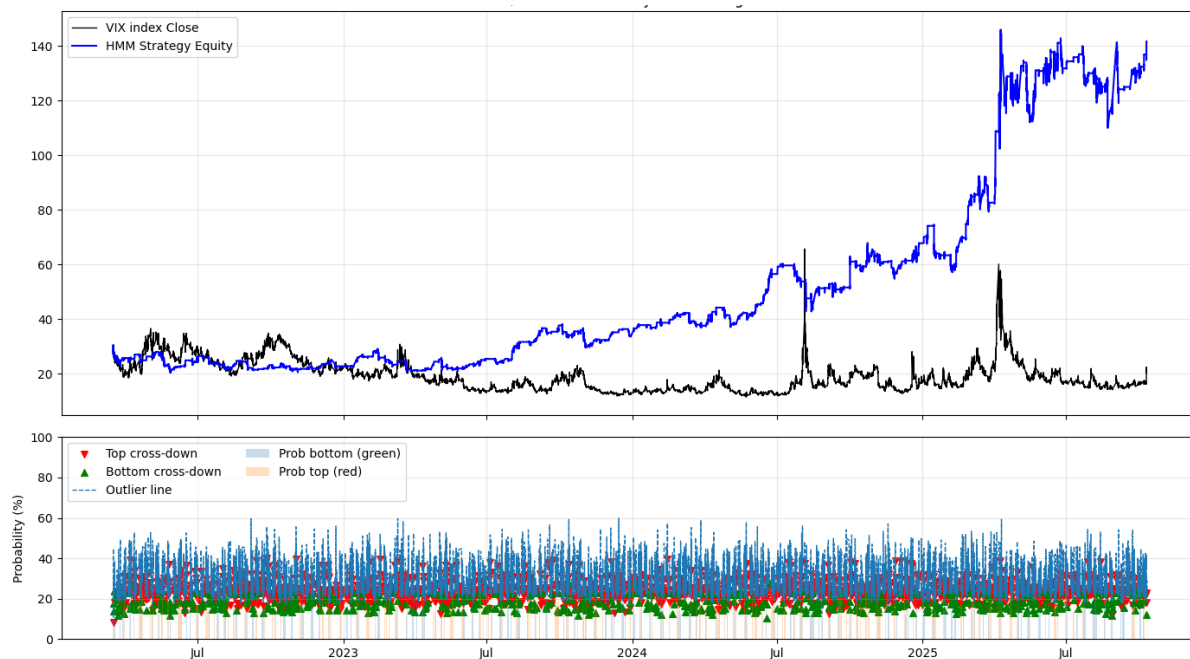


Figure 19: Resultats HMM Bayesian

13 Implémentation avec des réseaux de neurones

Notre [HMM](#) Nous renvoie donc maintenant de meilleures détections de régimes que le HMM classique, mais nous voulons maintenant un acteur qui nous permette de renvoyer avec précision des signaux de vente et d'achat. Car, bien que notre modèle produise de bons régimes, il subsiste des faux positifs. Nous voulons donc supprimer ces erreurs ; pour cela, nous allons utiliser une architecture composée de réseaux de neurones qui apprendront également des patterns dans le marché et se concatèneront avec les probabilités renvoyées par le HMM afin de pouvoir renvoyer une probabilité finale de hausse (ou de baisse) précise.

Dans notre architecture, nous utiliserons des [LSTM](#) et [MLP](#). Voici figure 20 la description détaillée de l'organisation de notre entraînement

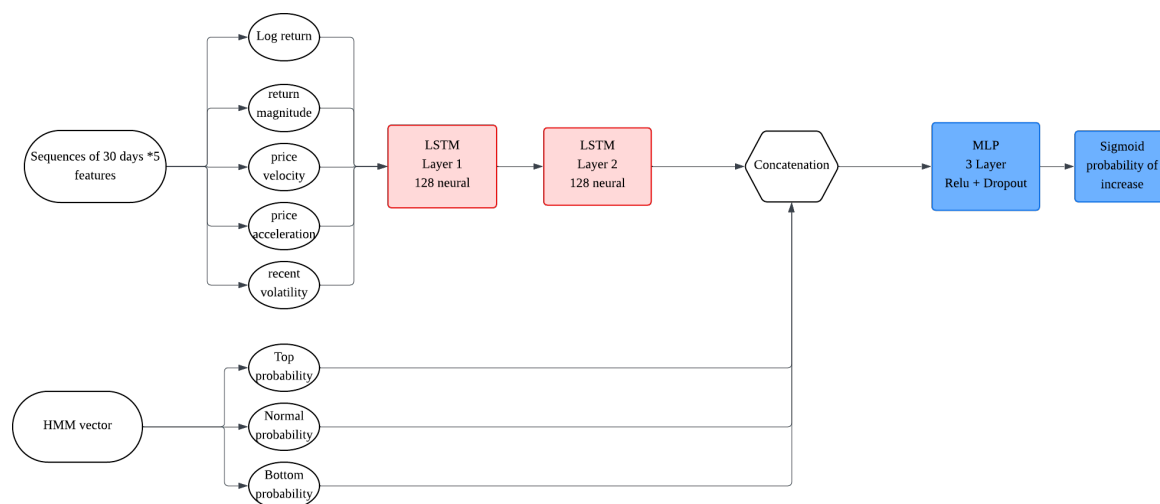


Figure 20: Architecture de l'entraînement

Notre HMM détaillé précédemment nous renvoie donc les 3 probabilités des régimes raffini.

Le [LSTM](#) contrairement à un réseau de neurone classique permet de comprendre les dépendances temporelles. Grâce à sa "mémoire" modélisée par ses différentes portes d'entrée, de sortie et d'oubli, il peut s'adapter et détecter des patterns de prix.

Le [MLP](#) servira de cerveau final, il prendra en entrée les états cachés renvoyés par le HMM et la détection du LSTM, il combinera ces 2 visions pour renvoyer une dernière prédiction robuste.

Pour entraîner ce modèle nous utilisons dans un premier temps des fonctions d'erreur simple telles que la [MSE](#) et [BCE](#) qui demandent seulement au modèle de prédire le prix. Les résultats à l'aide de ces fonctions étaient limités car ces fonctions d'erreurs traitent chaque erreur de la même manière, si la prédiction se trompe de 0.1% ou de 5% notre modèle supposera que nous avons réalisé la même erreur, ce qui n'est pas du tout optimal.

Nous avons donc finalement amélioré notre modèle en utilisant la fonction objectif **SharpeLoss** qui contrairement à la BCE vise à maximiser le rendement mais à ajuster au risque. Pour cela, notre fonction objectif calcule le [sharpe ratio](#). Elle a besoin de 2 valeurs :

- La confiance (y_i) : La probabilité de hausse donnée par le LSTM qui représente la

taille de notre position

- Les rendements (R_i): Le rendement reel du marche observe plus tard

Notre model calcule donc le rendement de la strategie

$$R_{\text{strat},i} = R_i * y_i$$

En ayant cela, nous allons maintenant calculer la moyenne et la volatilit  de ces rendements sur l'ensemble de notre batch (qui est le groupe de donnees actuel)

$$\mu = \frac{1}{n} \sum_{i=1}^n R_{\text{strat},i}$$

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (R_{\text{strat},i} - \mu)^2 + \varepsilon}$$

Le sharpe ratio est defini par

$$\text{Sharpe} = \frac{\mu}{\sigma}$$

Finalement, en Deep Learning la fonction perte cherche toujours a minimiser une valeur, ainsi comme nous voulons maximiser le sharpe ratio. Alors la fonction de perte sera l'inverse du sharpe ratio

Cette architecture nous permet d'ameliorer nos signaux, de les faire gagner en precision et reduire drastiquement les faux positifs. Ce modele ci prend donc moins de positions sur le marche mais prend des propositions plus precise et reflechi. En effet, il prend une position lorsque le LSTM et le HMM sont en accord. Nous pouvons voir cela sur la figure 21, notre robot prend position sur les confimed buy et sell mais si seulement le HMM envoie des signaux notre modeles les ignorent :

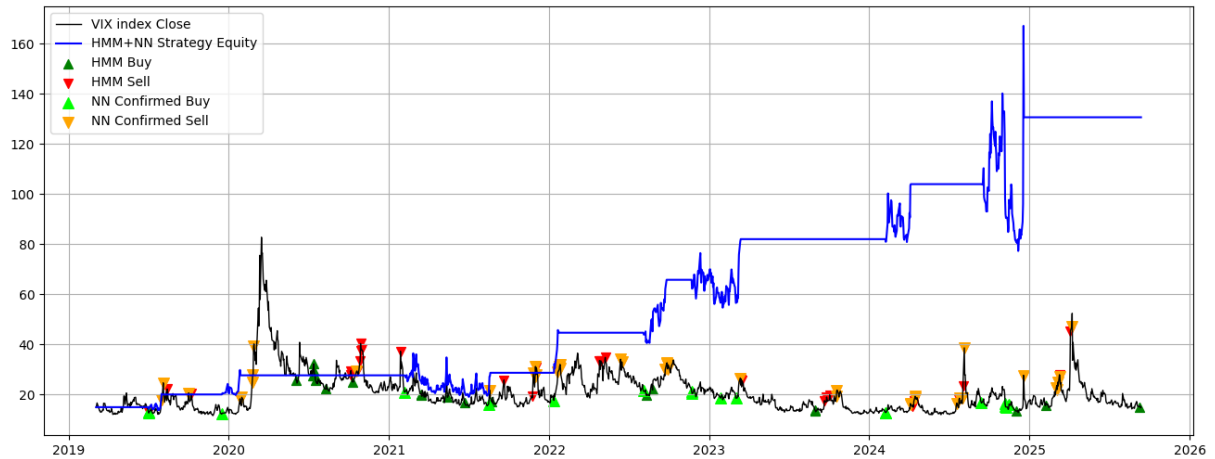


Figure 21: visualisation en période de test

14 Utilisation d'optuna pour optimiser l'entraînement

Pour réaliser l'entraînement précédent, nous avons utilisé des valeurs arbitraires des différents hyperparamètres permettant de réaliser un entraînement, tels que le nombre d'époques, le learning rate, le batch size ou encore la durée de la séquence.

Ne sachant pas quelles valeurs optimales nous devons renseigner pour que notre modèle apprenne bien, et ne voulant pas tout tester à la main car cela prendrait un temps fou, nous allons utiliser Optuna, qui a été créé pour automatiser l'optimisation d'hyperparamètres et qui va pouvoir tester à notre place toutes les valeurs, et nous indiquer quels hyperparamètres sont les plus importants pour que notre modèle ait de meilleurs résultats.

Nous avons donc lancé Optuna pour optimiser les hyperparamètres suivants :

- **Le nombre d'époques** : Une époque correspond à un passage complet de l'ensemble des données d'entraînement à travers le réseau de neurones. Une trop petite valeur peut provoquer de l'underfitting tandis qu'une trop grande de l'overfitting
- **Le Learning rate** : C'est la taille des pas que fait l'algorithme d'optimisation pour ajuster les poids du modèle afin de minimiser la perte (la SharpeLoss).
- **Le Batch size** : Le modèle traite les données par groupes, ceci permet de régler la taille des différents groupes.
- **La longueur de séquence** (training épisode) : C'est la "fenêtre de regard" en arrière que le LSTM utilise pour chaque prédiction.

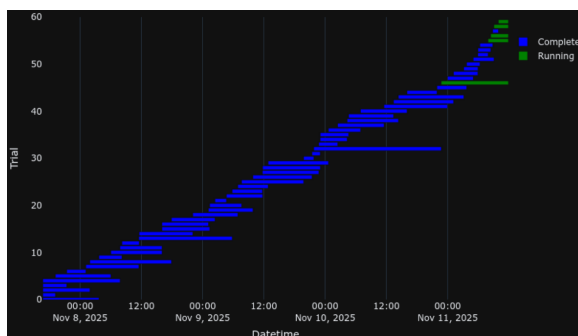


Figure 22: différents entraînements

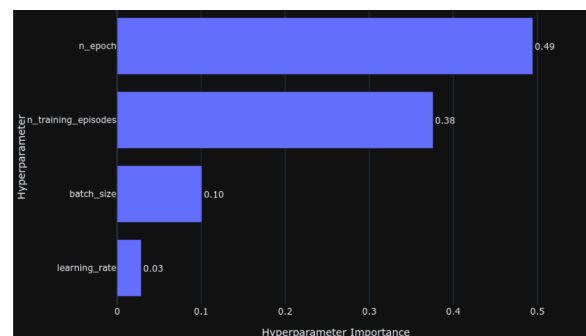


Figure 23: Importances des hyperparamètres

Comme le montre la figure 23, nous voyons que certains paramètres ont plus d'importance que d'autres ; ceci a permis à Optuna de pouvoir choisir en priorité quels hyperparamètres modifier lors des entraînements. Nous avons finalement retenu les valeurs suivantes pour nos différents hyperparamètres, car ces valeurs offraient un entraînement optimal :

Training_episodes = 90
 BATCH_SIZE = 128
 Learning_rate = 10^{-4}
 N_EPOCHS = 300

Nous pouvons voir ci-dessous une figure représentant les différentes equity curves lors de la période de test sur un intervalle de temps donné. Nous voyons qu'en changeant certains hyperparamètres, nos performances peuvent drastiquement changer..

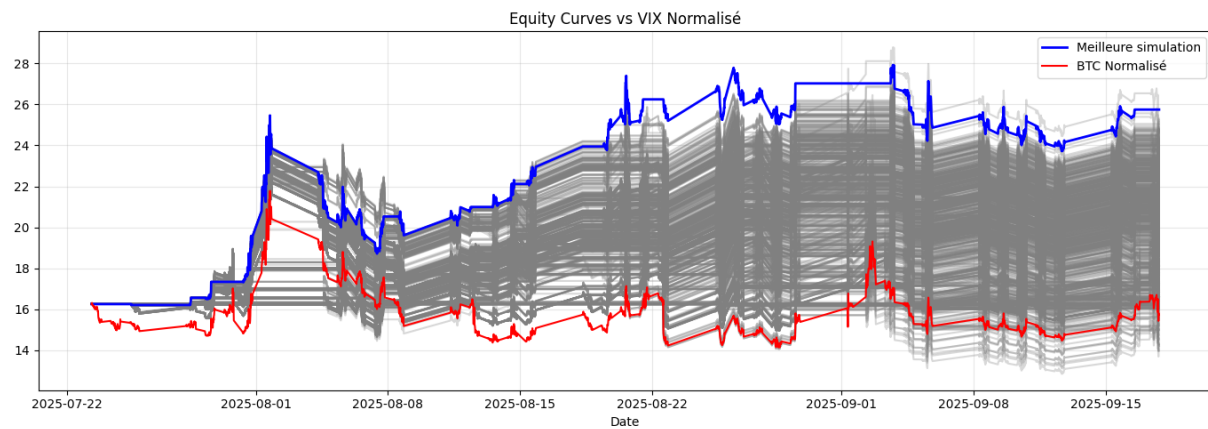


Figure 24: Visualisation des optimisations avec optuna

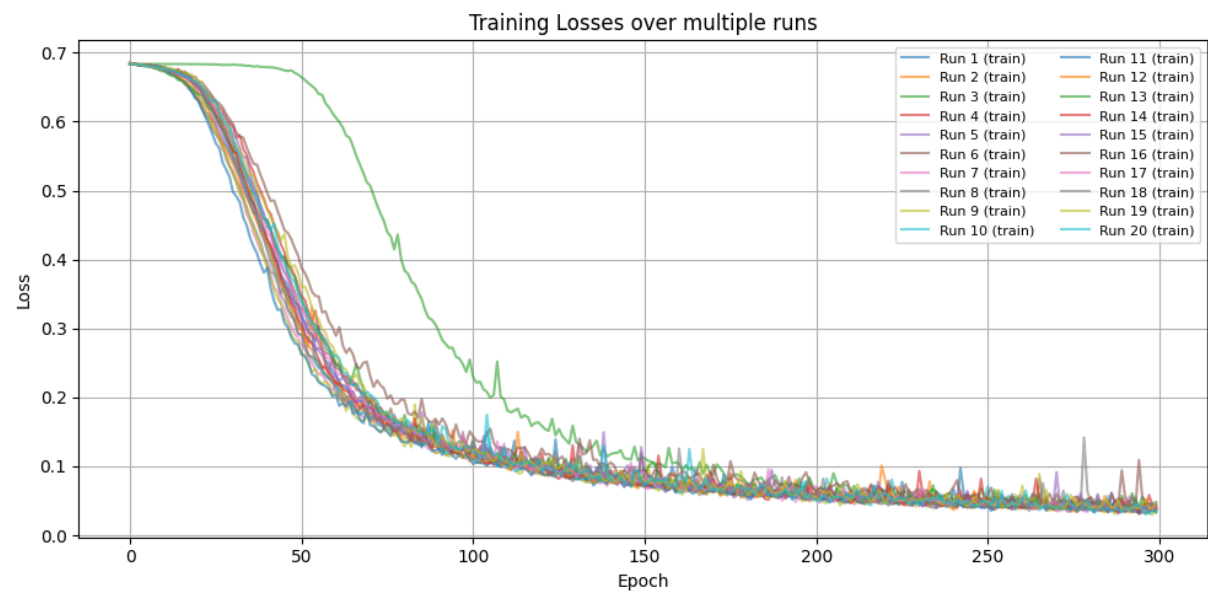


Figure 25: Evolution des loss

15 Entraînement

En ayant désormais nos modèles optimisés au mieux, nous relançons les entraînements, en faisant également varier la timeframe sur laquelle nous travaillons. Comme nous pouvons le voir sur la figure 23, nous séparons nos données en trois parties : entraînement / validation / test.

Nous avons entraîné différents modèles : HMM classique, Variational Bayesian HMM. De plus, nous avons modifié les données nourries en entrée de nos entraînements : entraînement avec seulement les données du VIX Index, entraînement avec les VIX Futures, le VVIX Index ou encore avec le S&P 500.

Nous avons également entraîné nos modèles sur différentes timeframes afin d'étudier complètement les performances et de pouvoir déterminer quelle timeframe utiliser.

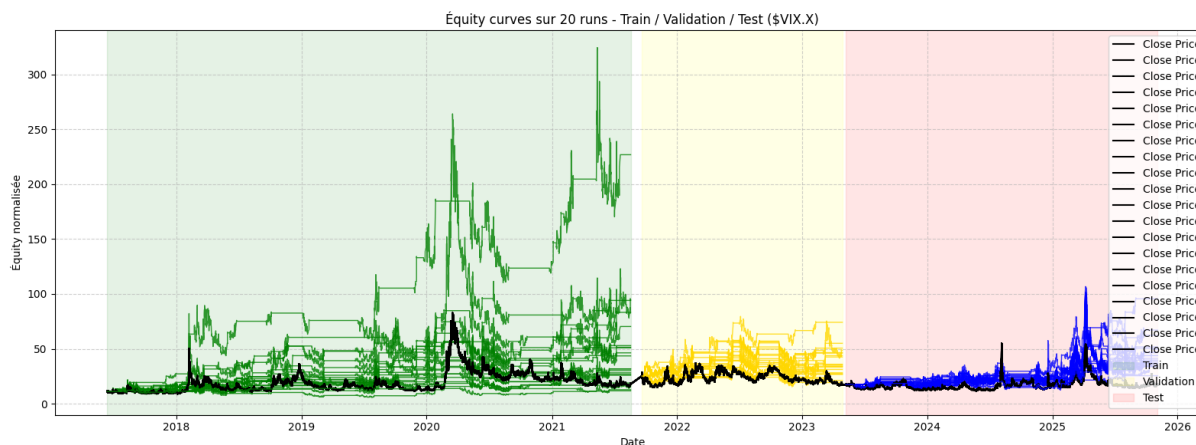


Figure 26: train/validation/test

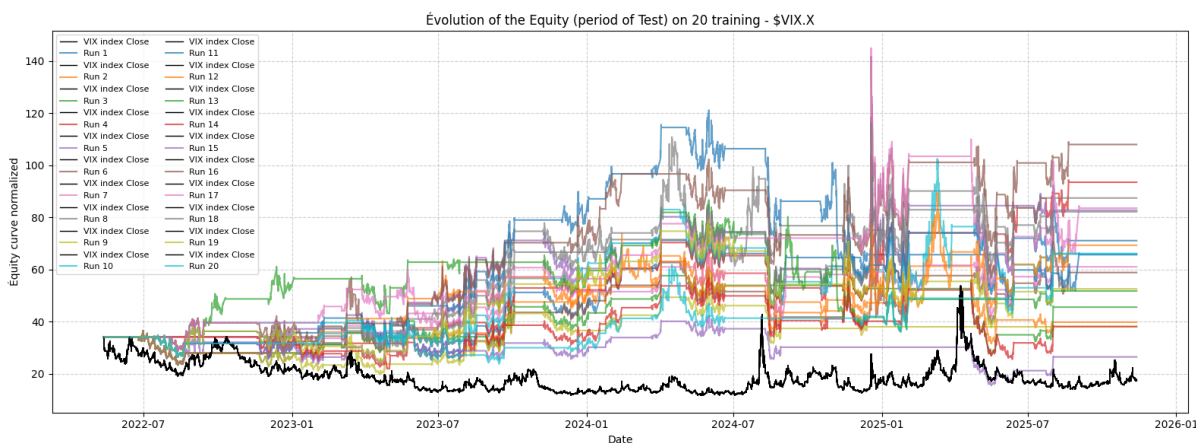


Figure 27: Periode de test sur 20 runs

16 Resultats

Après avoir entraîné nos différents modèles, nous analysons maintenant les résultats à l'aide de plusieurs critères, énoncés ci-dessous. Nous ne décrivons pas ici l'ensemble des modèles entraînés, car leur nombre est très élevé. Nous nous concentrerons uniquement sur ceux qui sont pertinents pour notre étude, ainsi que sur le modèle retenu pour une utilisation en temps réel.

Performances	Signification
Rendement total	rendement total sur toute la periode de test en pourcentage ou en dollars
Rendement annuel	rendement annuel en pourcentage ou en dollars
volatilite annuel	Représente la volatilité de la stratégie. Une valeur élevée indique des prises de position proches du hasard.
max drawdown	represente la perte maximale observee entre un sommet et un creux sur l'equity curve
drawdown moyen	moyenne de toute les pertes observees
timeframe	Duree d'une bougie
average profit per trade	le profit moyen d'un trade (en pourcentage ou en dollar)
average trade duration	la duree moyenne d'un trade

Nous avons donc entraine differents modeles sur differentes timeframes dont voici quelques resultats ci dessous. L'equity curve representant l'evolution du portefeuille est base sur le prix du VIX index mais nous aurions egalement pu la calculer en dollar puisque chaque achat represente un contrat future, qui est plus courant de s'exprimer en dollar.

17 Performances de la strategies en utilisant l'implementation aux reseaux de neurones

	2h timeframe trading	5 min timeframe trading
duration of the backtest	3 years	3years
total return	377 %	106 %
annual return	56.85 % / \$11,370	27 % / \$5,400
annual volatility	19 %	12 %
sharpe ratio	2.4	1.2
Maximum drawdown	30 %	26 %
Average drawdown	6 %	9 %
number of trades	17	456
average profit per trade	10.96 % / \$2,192	0.47 % / \$55.91
average trade duration	20 days	27 hours

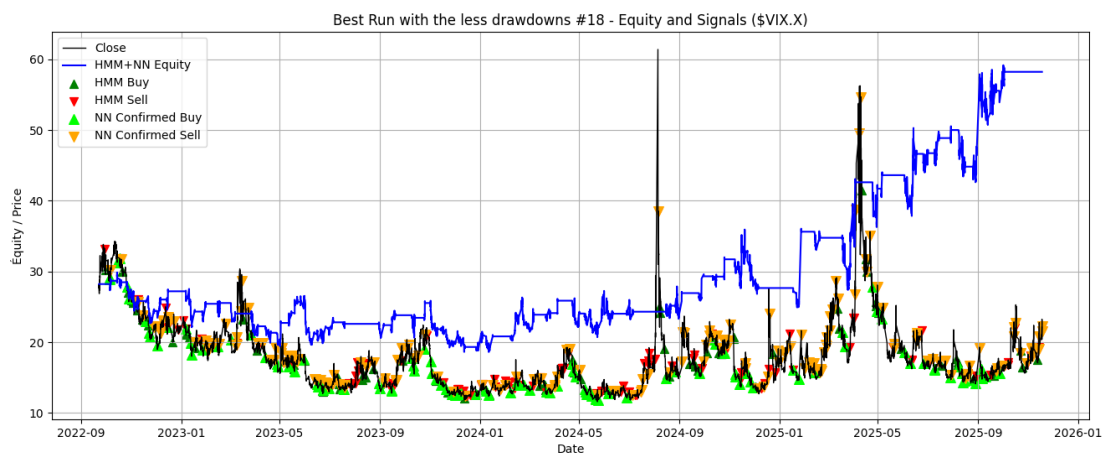


Figure 28: Backtest en timeframe 5 min

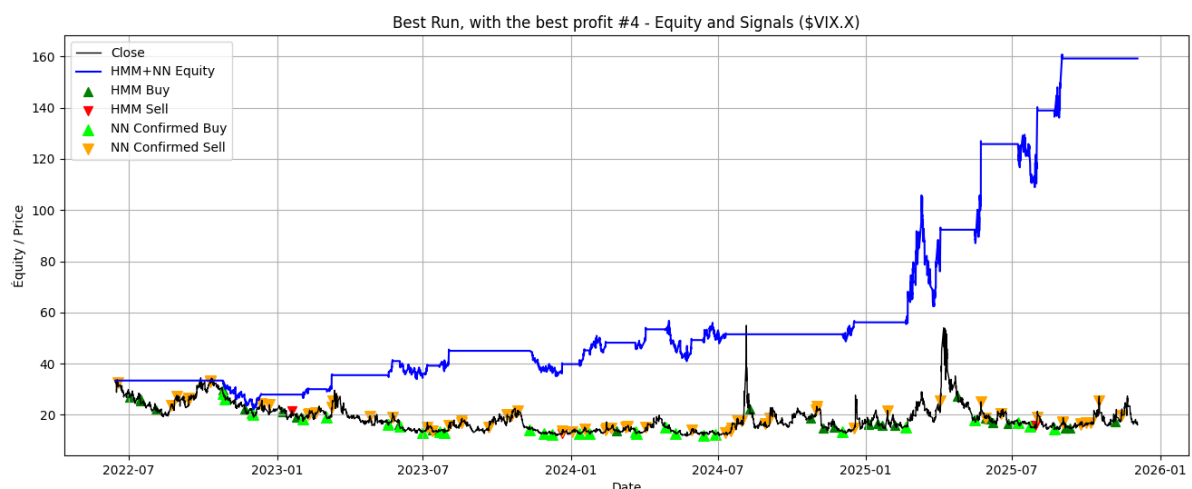


Figure 29: Backtest en timeframe 2h

18 Performances de la stratégie utilisant seulement le modèle de Markov caché

	2h timeframe trading	5 min timeframe trading
duration of the backtest	3 years	3 years
total return	247 %	345 %
annual return	38.85 % / \$6,970	53.47 % / \$9,520
annual volatility	19 %	9 %
sharpe ratio	2.39	1.46
Maximum drawdown	32 %	42 %
Average drawdown	7 %	19 %
number of trades	70	924
average profit per trade	9.25 % / \$1,912	0.18 % / \$36
average trade duration	13 days	20 hours

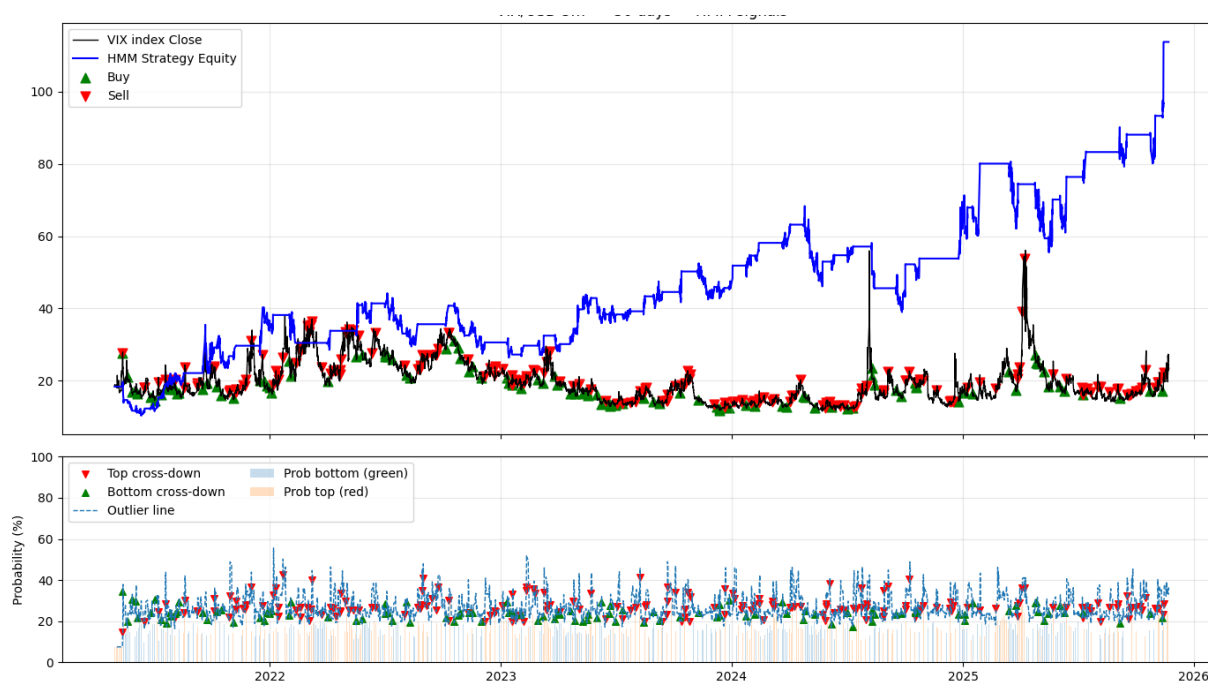


Figure 30: Resultats HMM Bayesian

Nos différents backtests nous ont offert des performances respectables. Certaines stratégies peuvent comporter de gros drawdowns, mais cela est cohérent avec les rendements réalisés. Nous avons évoqué précédemment que tenir une position trop longtemps peut coûter des frais, et c'est pour cette raison que, bien que les performances pour la timeframe de 2 h soient excellentes, nous utiliserons par la suite les modèles entraînés sur une timeframe de 5 minutes afin de réaliser du daily trading et ainsi éviter les frais.

19 Travail en temps reel

19.1 API TradeStation

Une fois que nous avons obtenu un model robuste il nous fallait maintenant le tester en temps reel pour pouvoir verifier que les performances soient semblables a la simulation

Pour realiser cela nous avons du travailler avec la plateforme de brokerage TradeStation, et utiliser l'[API](#) associe a un compte professionnel permettant d'accéder au marche et de pouvoir se positionner dessus.

Pour acceder aux donnees en temps reel nous n'avions qu'un compte pour 2 personnes, pour cela nous avons du creer une architecture permettant a plusieurs personne d'utiliser l'[API](#). Nous avons creer un environnement Docker pour deployer facilement l'API sur different serveurs. Docker a ete utilise pour eviter les problemes de compatibilite de versions entre les differents environnements linux

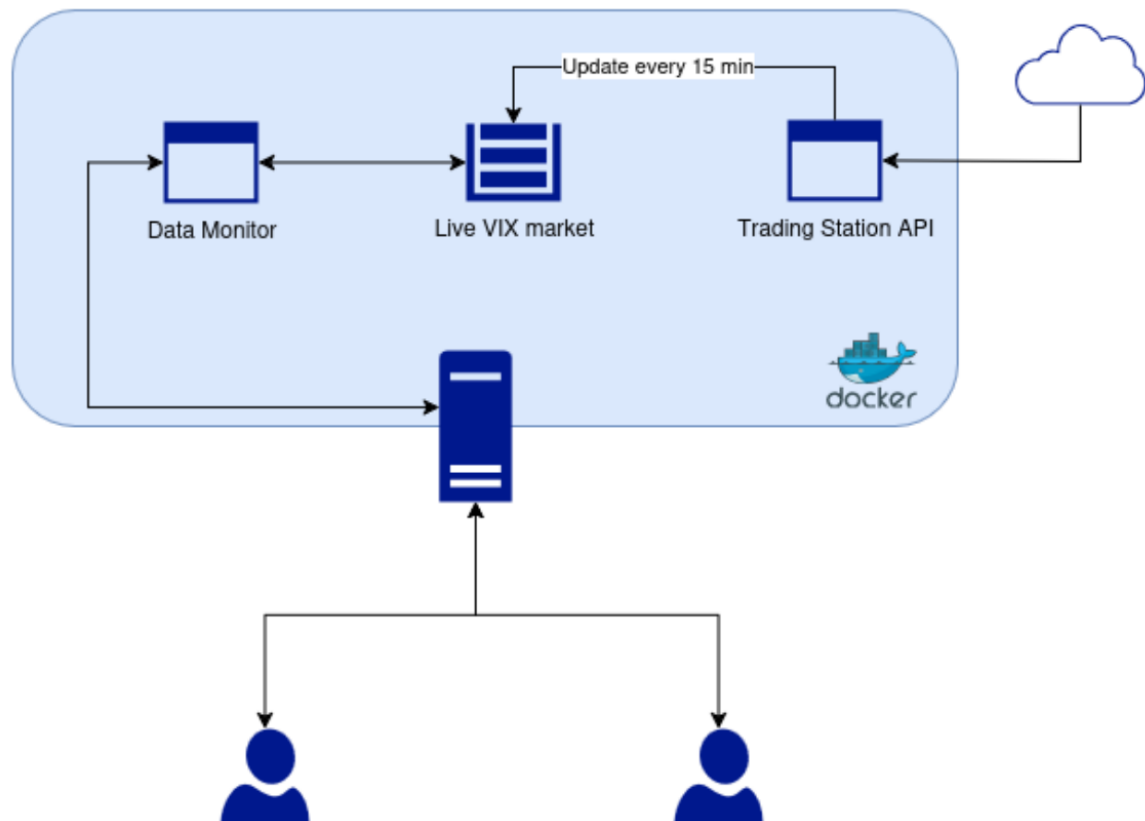


Figure 31: Environnement du serveur

Comme illustré dans la figure ci-dessus, lorsqu'il est actif, le service met à jour sa base de données via l'API TradeStation et synchronise automatiquement son horloge avec les serveurs TradeStation afin de garantir un signal en temps réel. Par ailleurs, la base de données du serveur gère les différents clients à l'aide de requêtes [SQL](#) pour éviter les conflits lors de la lecture des données. Le serveur intègre également un moniteur HTTP permettant de vérifier son bon fonctionnement.

Après avoir déployé l'API sur le serveur, une connexion sûre entre les différents ordinateurs du laboratoire devait être réalisée. Ainsi un autre membre de l'équipe a développé un second service côté client afin d'assurer une synchronisation rapide et stable avec le serveur. De plus, pour simplifier l'intégration du signal en flux continu dans nos scripts Python, une bibliothèque dédiée (illustrée à droite) a été conçue, permettant à tout programme Python d'accéder simultanément aux données en temps réel. Ceci est réalisé grâce à des threads d'arrière-plan qui se connectent automatiquement au service client et transmettent en continu le signal de marché entrant à l'agent.

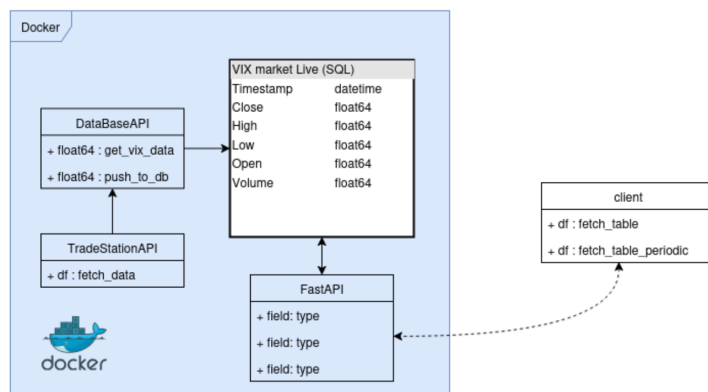


Figure 32: Serveur/client interface

19.2 Implémentation en temps réel

L'objectif du temps réel est de vérifier principalement que l'exécution des ordres se produise bien comme nous l'avons espéré en simulation. En effet, sur un marché réel, il peut y avoir des différences avec la simulation lors de l'achat d'un actif. En réel, par exemple, si l'on veut acheter un produit qui coûte \$100, il est possible de ne pouvoir l'acheter en réalité qu'à \$100,3, ce qui provoquerait une perte pour nous. Ceci est appelé le [slippage](#).

Ceci est un exemple, mais il faut que les performances en temps réel soient semblables à celles en simulation et que notre modèle puisse réagir à différentes erreurs simples inattendues qui peuvent survenir, telles qu'une coupure réseau, un crash du programme, ou un scénario sur le marché non prévu par la stratégie.

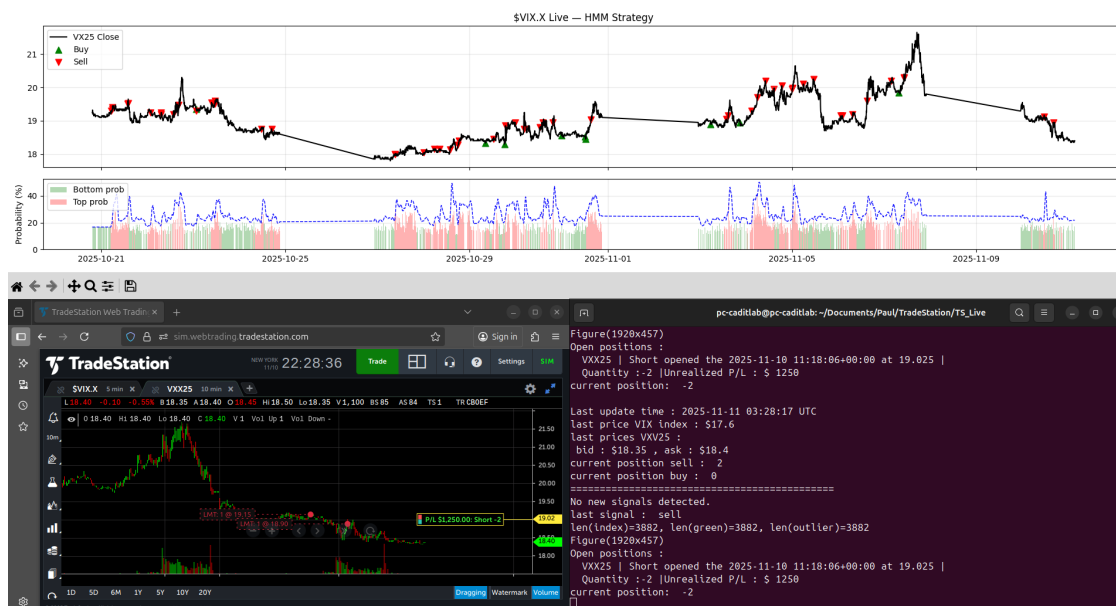


Figure 33: interface en temps réel

Nous avons donc lancé notre modèle en temps réel afin de pouvoir étudier ses performances. Ceci est représenté sur la figure 33. Les outils de l'API de TradeStation étaient très intuitifs et faciles à prendre en main. Cela nous a permis de développer notre code en Python simplement. Il fallait s'occuper de récupérer les données de prix sur un intervalle de temps passé ainsi que d'actualiser environ toutes les 10 secondes afin de récupérer les nouvelles données de prix.

Il fallait également prévoir la logique d'achat et de vente en fonction des signaux reçus, en respectant une certaine logique, car tous les signaux ne devaient pas être pris en compte. Notre programme fonctionnait comme nous l'attendions et respectait les performances que le backtest nous avait offertes.

20 Travail Future

Le modèle développé fonctionne bien, il répond au cahier des charges de pouvoir filtrer le marché. De plus, l'intégration aux réseaux de neurones améliore son efficacité.

Parallèlement à mon travail, un autre membre de l'équipe développait un outil de reinforcement learning, un [PPO](#) avec acteur et agent, et il peut donc être pertinent, pour un futur travail, d'intégrer le modèle de Markov caché en tant qu'acteur dans cette infrastructure.

De plus, pour améliorer la rapidité et la robustesse en temps réel, il aurait été pertinent de développer le code en C.

21 Conclusion

Durant ce stage, j'ai pu appliquer en détail des mathématiques complexes et étudier comment les mathématiques pouvaient être appliquées à la finance et, plus généralement, au Deep Learning.

Ce stage m'a permis de découvrir un domaine qui n'est pas forcément commun pour un étudiant en génie électrique. Ce domaine est la finance de marché. J'étais déjà curieux du monde de la finance avant de réaliser ce stage. Cette expérience m'a permis d'y plonger et de l'analyser plus en détail. J'ai également durant ce semestre réalisé mon PPH sur la question : "Le monde de la finance, un marché régit par les plus puissants ? ". J'ai pu discuter avec des professionnels qui m'ont éclairé sur la question.

J'ai également pu appliquer mes compétences acquises en cours d'analyse numérique, de traitement du signal et de mathématiques. Pouvoir créer soi-même un modèle demande une grande compréhension des outils utilisés, et cela m'a permis de distinguer des aspects du machine Learning que je n'avais pas forcément pris en compte. Pouvoir travailler avec des personnes expérimentées sur ces sujets complexes est extrêmement enrichissant.

Enfin, ce stage a été réalisé en Corée du Sud, à Séoul, et cela a été pour moi une expérience forte. Découvrir le monde du travail coréen, qui est très différent de celui de la France, m'a permis de m'ouvrir, et j'ai également pu découvrir la culture coréenne, ce qui a été un dépaysement total.

Cette expérience m'a permis de travailler avec des personnes de différents pays et ainsi d'améliorer mon niveau d'anglais qui est un atout phare dans le monde du travail aujourd'hui.

Acronymes

API Application Programming Interface

BCE Binary Cross Entropy

FPGA Field Programmable Gate Array

HMM Hidden Markov Model

IA Intelligence Artificielle

LLM Large Language Model

LSTM Long Short Term Memory

MSE Mean Squared Error

MLP Multi-Layer Perceptron

SNU Seoul National University

PPO Proximal Proximity Agent

SLAM Simultaneous Localization And Mapping

SQL Structured Query Language

Glossaire

drawdowns Un drawdown correspond à la perte d'investissement subie entre un point haut et un point bas. . [23](#), [37](#)

Futures les contrats futures sont des instruments financiers qui permettent aux acteurs du marché de compenser ou d'assumer le risque de variation du prix d'un actif au fil du temps.. [10](#), [23](#), [37](#)

sharpe ratio Le sharpe ratio indique si les rendements d'un portefeuille sont attribuables à des décisions judicieuses ou à la chance. Il compare le rendement d'un investissement à son risque. Plus le sharpe est eleve signifie plus la strategie est robuste . [24](#), [37](#)

slippage Le slippage désigne la différence entre le prix attendu d'une transaction et le prix auquel elle est exécutée. Le slippage peut survenir à tout moment, mais il est plus fréquent en période de forte volatilité. . [33](#), [37](#)

22 References

References

- [1] Monteiro, Tiago. 2024. *AI-Powered Energy Algorithmic Trading: Integrating Hidden Markov Models with Neural Networks*. arXiv preprint arXiv:2407.19858v5.
- [2] Nguyen, Nguyet, and Dung Nguyen. 2018. *Global Stock Selection with Hidden Markov Model*. Youngstown, OH: Department of Mathematics and Statistics, Youngstown State University; Venice, FL: Ned Davis Research Group.
- [3] Oelschläger, Lennart, Timo Adam, and Rouven Michels. 2020. *fHMM: Hidden Markov Models for Financial Time Series in R*. Bielefeld: Bielefeld University; St Andrews: University of St Andrews.
- [4] Song, Xingyao, Wenyu Chen, and Junyi Lu. 2021. *Research on Financial Stock Market Prediction Based on the Hidden Quantum Markov Model*. Sydney: Business School, The University of Sydney; Chengdu: University of Electronic Science and Technology of China.
- [5] Yuan, Yuan. 2019. *Market Regime Identification Using Hidden Markov Model*. London: Department of Computer Science, University College London.